Conducting computational investigations

A practical guide to evidence, experiments and decisions

The purpose of an investigation is to reduce uncertainty about a question. Judge progress by what the evidence establishes and which decision it changes. Better scores, more code and more detailed explanations are useful only in relation to that purpose.

Frame the problem and make progress testable

1. Choose the simplest useful representation

Define the result you need, the assumptions it requires and what would count as convincing evidence. Separate tasks with different evidence needs. Recovering order, identity or relationships may be possible before recovering exact shape or appearance. Start with the representation that exposes the strongest constraints: a graph, sequence, conservation law or small numerical model. Add detail when it resolves a demonstrated limitation.

2. Establish a small trustworthy reference

Use a modest amount of careful human annotation or direct measurement to check automation. Include clear cases, representative difficult cases and explicit unknowns. Preserve the original source and distinguish a measurement, its interpretation and a model prediction. Several features may belong to one object; repeated interpretations of the same source are not independent evidence. Allow abstention when the source cannot support the requested judgment.

3. Make each experiment distinguish explanations

Before running it, name the uncertainty, competing explanations, intervention and outcomes that would change your decision. Prefer the smallest informative test. Compare against a simple baseline on the same observations, including misses and abstentions. Use withheld regions, cases or views to test prediction. A synthetic fixture intended to validate a repair should reproduce the original failure; check that the old method actually fails it. Passing convenient tests does not establish reliability on the motivating problem.

4. Verify the computation before enlarging it

For inverse problems, test recovery from data generated with known answers. Evaluate the true answer directly, then test perturbed starts and known correspondence before adding ambiguity. Judge recovery up to symmetries or equivalent parameterizations. Check that each parameter affects the actual calculation; inspect units, bounds, coupling and sampled combinations. A large parameter list or trial count does not demonstrate effective exploration.

5. Preserve working knowledge and earlier successes

Keep source provenance, immutable results and candid failure reports. Maintain one short current record: **claim, premises, evidence, counterexample, next test**. When extending or replacing a method, replay earlier successful predictions on the same cases. Changes to normalization or parameter meaning can destroy a local success without an obvious code defect. Report exclusions and changed denominators explicitly. Preserve alternatives until evidence distinguishes them.

Diagnose the failure and choose the next action

Distinguish four explanations for a poor result

Explanation	Smallest useful check	Appropriate response
Search failure: the procedure misses a valid, representable answer.	Evaluate known truth; test nearby starts, active parameters and approximation errors.	Repair search or numerics before scaling.
Model inadequacy: the model excludes necessary behavior.	Show a contradiction; relax one meaningful assumption and recheck constraints.	Revise the model. Failed search alone does not establish inadequacy.
Wrong objective: a good score permits a wrong answer.	Check the same candidates against independent correctness and essential constraints.	Change the target or acceptance test.
Insufficient identification: distinct answers fit the measurements.	Find an observation on which coherent alternatives predict different outcomes.	Report uncertainty or obtain that evidence.

Several mechanisms may coexist. Keep the diagnosis provisional until a test distinguishes them. An unexplained stall is not evidence for any one explanation.

Use warning signs as decision gates

- **Stop adoption** when a better score accompanies worse independent correctness, violates an essential constraint or depends on predictions validating themselves. Expose the tradeoff and diagnose it before accepting the result.
- **Stop expansion** when a small known-answer inverse test or a basic measurement check fails. Establish why it fails before funding the larger version.
- **Reconsider the approach** when complexity grows while the same independent counterexample survives. Require a new causal hypothesis and a discriminating test before repeating the campaign.
- **Change representation or acquire evidence** when local decisions cannot coexist globally, results are unstable under reasonable perturbations, or equally good fits imply different answers. Test which mechanism applies first.
- **Continue** when credible evidence reduces a stated uncertainty, essential earlier successes survive, and the next experiment has a clear decision purpose.

Set task-specific tolerances and essential constraints before evaluating results. These gates concern evidence, not an arbitrary number of iterations. Enabling work can be worthwhile; name the obstacle it removes and verify that removal.

Close every iteration with a decision

Record what changed, what remains contradicted or unknown, and the smallest test that could change the conclusion. Require this account from an AI agent before another substantial round. Software tests check implementation; scientific claims require evidence. If the question remains undecidable, deliver a qualified result and name the additional observation needed.

Basis: Retrospective, chapter 7 and Future project methods. The recommendations are lessons from an investigation, not guarantees that every uncertainty is avoidable.